

05_hybrid_integration

August 14, 2026

1 Part 4: Hybrid System Integration

```
[12]: from sklearn.preprocessing import MinMaxScaler
import numpy as np
import pandas as pd
```

```
[13]: u1_base_mat = pd.read_csv("../data/u1.base",
    ↪names=["user_id", "item_id", "rating", "timestamp"], sep="\t", )
u1_test_mat = pd.read_csv("../data/u1.test",
    ↪names=["user_id", "item_id", "rating", "timestamp"], sep="\t")
```

```
[14]: # Load metadata from u.item
u_data = pd.read_csv("../data/u.data", sep="\t",
    ↪names=["user_id", "item_id", "rating", "timestamp"])

u_item_cols = [
    "movie_id", "movie_title", "release_date", "video_release_date", "IMDb_URL",
    "unknown", "Action", "Adventure", "Animation", "Children", "Comedy", "Crime",
    "Documentary", "Drama", "Fantasy", "Film-Noir", "Horror", "Musical", "Mystery",
    "Romance", "Sci-Fi", "Thriller", "War", "Western"
]

u_item = pd.read_csv("../data/u.item", sep="|", header=None, names=u_item_cols,
    ↪encoding="latin-1")
# Drop duplicate movie titles, keeping the first occurrence
u_item = u_item.drop_duplicates(subset="movie_title", keep="first").
    ↪reset_index(drop=True)

genres_cols = [
    "Action", "Adventure", "Animation", "Children", "Comedy", "Crime", "Documentary",
    "Drama", "Fantasy", "Film-Noir", "Horror", "Musical", "Mystery", "Romance",
    "Sci-Fi", "Thriller", "War", "Western"
]
```

```
[15]: # Distinguish users by of ratings
users = u1_base_mat.groupby("user_id").size()
user_ratings = users.reset_index()
```

```
user_ratings.columns = ["user_id", "rating_count"]
```

```
[16]: # Parse the release year from movie titles
u_item["release_year"] = u_item["movie_title"].str.extract(r"\((\d{4})\)").
    ↪astype(float)

# Genre list column
genre_list = []
for i, row in u_item.iterrows():
    genres = []
    for g in genres_cols:
        if int(row[g]) == 1:
            genres.append(g)
    genre_list.append(genres)
u_item["genre_list"] = genre_list

# Create a synthetic runtime column
np.random.seed(0)
u_item["runtime_min"] = np.random.choice(
    [60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, 200, np.
    ↪nan],
    size=len(u_item),
    p=[0.01, 0.03, 0.08, 0.15, 0.18, 0.18, 0.15, 0.08, 0.04, 0.03, 0.02, 0.015,
    ↪0.01, 0.005, 0.004, 0.016]
)

runtime_cat = []
for rt in u_item["runtime_min"]:
    if pd.isna(rt):
        runtime_cat.append("unknown")
    elif rt < 90:
        runtime_cat.append("short")
    elif rt <= 120:
        runtime_cat.append("medium")
    else:
        runtime_cat.append("long")
u_item["runtime_category"] = runtime_cat
```

1.0.1 1. Implement decision logic:

```
[17]: # Decision logic:
weights = []
for user in user_ratings.itertuples(index=False):
    userid = user.user_id
    ratings = user.rating_count

    # New users (no ratings / cold-start): Use knowledge-based only
```

```

if ratings == 0:
    collab_filter_weight = 0.0
    knowledge_filter_weight = 1
# Users with <5 ratings: Knowledge-based primary (80%) + CF supplement (20%)
elif ratings < 5:
    collab_filter_weight = 0.2
    knowledge_filter_weight = 0.8
# Users with 5-20 ratings: Balanced hybrid (CF 60% + KB 40%)
elif ratings <= 20:
    collab_filter_weight = 0.6
    knowledge_filter_weight = 0.4
# Users with >20 ratings: CF primary (CF 80% + KB 20%)
else:
    collab_filter_weight = 0.8
    knowledge_filter_weight = 0.2

weights.append({
    "user_id": userid,
    "rating_count": ratings,
    "collab_filter_weight": collab_filter_weight,
    "knowledge_based_weight": knowledge_filter_weight,
})

hybrid_filter_weight = pd.DataFrame(weights)

```

```
[18]: hybrid_filter_weight.head()
```

```
[18]:
```

	user_id	rating_count	collab_filter_weight	knowledge_based_weight
0	1	135	0.8	0.2
1	2	40	0.8	0.2
2	3	28	0.8	0.2
3	4	14	0.6	0.4
4	5	91	0.8	0.2

1.0.2 2. Create score combination function:

```

[19]: def combine_scores(user_id, cf_df, kb_df, hybrid_weights):

    # ---- Get weights from your hybrid_filter_weight table ----
    row = hybrid_weights.loc[hybrid_weights["user_id"] == user_id]
    if row.empty:
        w_cf, w_kb = 0.0, 1.0    # default for new user
    else:
        w_cf = float(row["collab_filter_weight"].values[0])
        w_kb = float(row["knowledge_based_weight"].values[0])

    # ---- Convert CF & KB predictions to dictionaries (item_id: score) ----

```

```

    cf_scores = dict(zip(cf_df["item_id"], cf_df["pred_rating"])) if not cf_df.
    ↪empty else {}
    kb_scores = dict(zip(kb_df["item_id"], kb_df["kb_score"])) if not kb_df.
    ↪empty else {}

    # ---- Normalize CF predictions to 0-100 scale ----
    if cf_scores:
        scaler = MinMaxScaler(feature_range=(0, 100))
        cf_values = np.array(list(cf_scores.values())).reshape(-1, 1)
        cf_scaled = scaler.fit_transform(cf_values).flatten()
        cf_scores = dict(zip(cf_scores.keys(), cf_scaled))

    # ---- Weighted combination with fallback ----
    combined_scores = {}
    all_items = set(cf_scores.keys()) | set(kb_scores.keys())

    for item in all_items:
        cf_val = cf_scores.get(item)
        kb_val = kb_scores.get(item)

        if cf_val is not None and kb_val is not None:
            combined_scores[item] = w_cf * cf_val + w_kb * kb_val
        elif cf_val is not None:      # KB failed → fallback to CF
            combined_scores[item] = cf_val
        elif kb_val is not None:      # CF failed → fallback to KB
            combined_scores[item] = kb_val
        else:
            combined_scores[item] = 0

    return combined_scores

```

1.0.3 3. Implement fallback strategies:

```

[20]: def hybrid_recommend(user_id, cf_predictions_dict, kb_predictions_dict,
    ↪hybrid_weights, popularity_scores):

    cf_df = cf_predictions_dict.get(user_id, pd.DataFrame(columns=["item_id",
    ↪"pred_rating"]))
    kb_df = kb_predictions_dict.get(user_id, pd.DataFrame(columns=["item_id",
    ↪"kb_score"]))

    # --- Case 1: both empty → use popularity fallback ---
    if cf_df.empty and kb_df.empty:
        print(f" CF & KB unavailable for user {user_id} → using Popularity
    ↪fallback")
        return popularity_scores

```

```

# --- Case 2: only CF available ---
if not cf_df.empty and kb_df.empty:
    print(f" KB failed for user {user_id} → using CF only")
    return dict(zip(cf_df["item_id"], cf_df["pred_rating"]))

# --- Case 3: only KB available ---
if cf_df.empty and not kb_df.empty:
    print(f" CF failed for user {user_id} → using KB only")
    return dict(zip(kb_df["item_id"], kb_df["kb_score"]))

# --- Case 4: both available → combine normally ---
combined = combine_scores(user_id, cf_df, kb_df, hybrid_weights)
return dict(sorted(combined.items(), key=lambda x: x[1], reverse=True))

```

1.0.4 Optional: Popularity-Based Fallback

```

[21]: popularity_scores = (
    u1_base_mat.groupby("item_id")
    .agg(avg_rating=('rating', 'mean'), num_ratings=('rating', 'count'))
    .assign(popularity_score=lambda x: x["avg_rating"] * np.
    ↪ log1p(x["num_ratings"]))
    .sort_values("popularity_score", ascending=False)
    .head(50)
    .to_dict()["popularity_score"]
)

```

```

[22]: # -----
# Helper: Create CF Predictions Dictionary
# -----
# Example CF predictions (replace with your actual CF output if available)
# Here we simulate some CF results
cf_pred_df = (
    u1_test_mat.copy()
    .groupby(["user_id", "item_id"])["rating"]
    .mean()
    .reset_index()
    .rename(columns={"rating": "pred_rating"})
)

# Dictionary form: {user_id: DataFrame(item_id, pred_rating)}
cf_predictions_dict = {
    user: df[["item_id", "pred_rating"]].reset_index(drop=True)
    for user, df in cf_pred_df.groupby("user_id")
}

# -----

```

```

# Create Personalized Knowledge-Based Predictions
# -----
genres_cols = [
    "Action", "Adventure", "Animation", "Children", "Comedy", "Crime", "Documentary",
    "Drama", "Fantasy", "Film-Noir", "Horror", "Musical", "Mystery", "Romance",
    "Sci-Fi", "Thriller", "War", "Western"
]

def get_user_genre_profile(user_id, base_mat, item_df, genres_cols):
    """Compute genre preference weights for a given user."""
    user_movies = base_mat[base_mat["user_id"] == user_id]
    rated_movies = item_df[item_df["movie_id"].isin(user_movies["item_id"])]
    if rated_movies.empty:
        return pd.Series(1, index=genres_cols) # equal preference for new users

    rated_movies = rated_movies.merge(
        user_movies[["item_id", "rating"]],
        left_on="movie_id",
        right_on="item_id",
        how="left"
    )
    genre_weights = rated_movies[genres_cols].multiply(rated_movies["rating"],
axis=0).sum()
    genre_weights = genre_weights / (genre_weights.max() if genre_weights.max()
    > 0 else 1)
    return genre_weights

def compute_kb_predictions(base_mat, item_df, genres_cols):
    """Generate KB predictions for all users based on their genre preferences.
    """
    kb_dict = {}
    for user in base_mat["user_id"].unique():
        prefs = get_user_genre_profile(user, base_mat, item_df, genres_cols)
        item_df["kb_score"] = (item_df[genres_cols] * prefs).sum(axis=1) * 20
    # 0-100 scale
    kb_dict[user] = item_df[["movie_id", "kb_score"]].
    rename(columns={"movie_id": "item_id"})
    return kb_dict

kb_predictions_dict = compute_kb_predictions(u1_base_mat, u_item.copy(),
genres_cols)

# -----
# Combine CF + KB Scores with Weight Logic

```

```

# -----
def combine_scores(user_id, cf_df, kb_df, hybrid_weights):
    """Combine CF and KB scores using weights."""
    row = hybrid_weights.loc[hybrid_weights["user_id"] == user_id]
    if row.empty:
        w_cf, w_kb = 0.0, 1.0
    else:
        w_cf = float(row["collab_filter_weight"].values[0])
        w_kb = float(row["knowledge_based_weight"].values[0])

    cf_scores = dict(zip(cf_df["item_id"], cf_df["pred_rating"])) if not cf_df.
    ↪empty else {}
    kb_scores = dict(zip(kb_df["item_id"], kb_df["kb_score"])) if not kb_df.
    ↪empty else {}

    if cf_scores:
        scaler = MinMaxScaler(feature_range=(0, 100))
        cf_values = np.array(list(cf_scores.values())).reshape(-1, 1)
        cf_scaled = scaler.fit_transform(cf_values).flatten()
        cf_scores = dict(zip(cf_scores.keys(), cf_scaled))

    combined_scores = {}
    all_items = set(cf_scores.keys()) | set(kb_scores.keys())

    for item in all_items:
        cf_val = cf_scores.get(item)
        kb_val = kb_scores.get(item)
        if cf_val is not None and kb_val is not None:
            combined_scores[item] = w_cf * cf_val + w_kb * kb_val
        elif cf_val is not None:
            combined_scores[item] = cf_val
        elif kb_val is not None:
            combined_scores[item] = kb_val
        else:
            combined_scores[item] = 0

    return combined_scores

# -----
# Fallback Logic (Always return recommendations)
# -----
def hybrid_recommend(user_id, cf_dict, kb_dict, hybrid_weights,
    ↪popularity_scores):
    """Generate final hybrid recommendations with fallback."""
    cf_df = cf_dict.get(user_id, pd.DataFrame(columns=["item_id",
    ↪"pred_rating"]))

```

```

kb_df = kb_dict.get(user_id, pd.DataFrame(columns=["item_id", "kb_score"]))

if cf_df.empty and kb_df.empty:
    print(f" CF & KB unavailable for user {user_id} → using Popularity_
↪fallback")
    return popularity_scores

if not cf_df.empty and kb_df.empty:
    print(f" KB failed for user {user_id} → using CF only")
    return dict(zip(cf_df["item_id"], cf_df["pred_rating"]))

if cf_df.empty and not kb_df.empty:
    print(f" CF failed for user {user_id} → using KB only")
    return dict(zip(kb_df["item_id"], kb_df["kb_score"]))

combined = combine_scores(user_id, cf_df, kb_df, hybrid_weights)
return dict(sorted(combined.items(), key=lambda x: x[1], reverse=True))

# -----
# Popularity-Based Fallback (Top movies)
# -----
popularity_scores = (
    u1_base_mat.groupby("item_id")
    .agg(avg_rating=('rating', 'mean'), num_ratings=('rating', 'count'))
    .assign(popularity_score=lambda x: x["avg_rating"] * np.
↪log1p(x["num_ratings"]))
    .sort_values("popularity_score", ascending=False)
    .head(50)
    .to_dict()["popularity_score"]
)

# -----
# TEST HYBRID RECOMMENDER
# -----
user_id = 10 # try different user IDs

final_recommendations = hybrid_recommend(
    user_id,
    cf_predictions_dict,
    kb_predictions_dict,
    hybrid_filter_weight,
    popularity_scores
)

print(f"\n Top 10 Hybrid Recommendations for User {user_id}:")

```

```

for item, score in list(final_recommendations.items())[:10]:
    title = u_item.loc[u_item["movie_id"] == item, "movie_title"].values
    title = title[0] if len(title) > 0 else f"Movie {item}"
    print(f"{title:50s} → {score:.2f}")

```

Top 10 Hybrid Recommendations for User 10:

Gone with the Wind (1939)	→ 87.24
Casablanca (1942)	→ 87.24
Stand by Me (1986)	→ 86.87
2001: A Space Odyssey (1968)	→ 86.74
Fargo (1996)	→ 86.28
Braveheart (1995)	→ 86.16
Godfather, The (1972)	→ 85.84
Sling Blade (1996)	→ 85.49
Wizard of Oz, The (1939)	→ 85.22
African Queen, The (1951)	→ 84.87

[]: